Data Augmentation for Thin-Film Thickness Regression: Comparing Mixup, kNN-Limited Mixing and RegMix on Simulated Spectra

Nisarg Telang Shashank Shelke Yogya Suryawanshi

Department of Computer Engineering (Regional Language)
Pimpri Chinchwad College of Engineering, Pune

Abstract

RegMix is a data augmentation method for regression that mixes each training example only with its nearest neighbours, using a reinforcement-learning controller to choose how many. Its original evidence rests on a competition dataset that is hard to obtain outside Korea, so we re-examine it on an open, physics-simulated stand-in: reflectance spectra of a four-layer silicon nitride and silicon dioxide film, with 226 features and four thickness labels. We implement RegMix, Mixup, Manifold Mixup, Global kNN and Global Distance in one JAX code base. Over ten paired seeds RegMix reaches 13.76 nm test RMSE, against 13.90 nm with no augmentation and 14.01 to 14.18 nm with unrestricted Mixup. It beats Mixup significantly after correction for multiple comparisons, but its 1% gain over no augmentation does not survive correction, and a tuned Global Distance baseline is statistically indistinguishable. A random policy gives no gain, yet three controllers trained from different seeds agree on which examples get which neighbourhood size no more often than chance (10.2 to 10.7%, against 9.3 to 9.4% expected by chance) while reaching similar error. The benefit therefore appears to come from the overall distribution of mixing ranges and not from a per-example assignment. Absolute errors are not comparable with the original paper because the data differ.

1 Introduction

Data augmentation is cheap when labels survive the transformation, and in regression they often do not. Flipping an image keeps its class, but perturbing a spectrum or sensor reading can shift a real-valued target by an unknown amount.

Mixup (Zhang et al., 2018) avoids this by mixing two examples and their labels with the same linear weight. For classification the mixed label stays near a few discrete classes. With continuous labels, mixing two distant inputs can produce a label far from the true one, because the underlying function is rarely linear over long distances (Hwang & Whang, 2021).

RegMix (Hwang & Whang, 2021) limits mixing to each example's k nearest neighbours and learns k per example with a reinforcement-learning controller. The controller is rewarded by the validation loss of a model trained on the mixed data. The original paper reports gains over several Mixup variants on semiconductor spectrum data.

That evidence rests on a proprietary dataset and a competition dataset that is hard to obtain from outside Korea. We therefore ask whether the idea holds up in an independent, fully open setting. This paper makes four contributions:

1. **An open simulator.** A transfer-matrix generator for four-layer thin-film reflectance spectra with the same input and label shapes as the competition data (226 spectral features, 4 layer thicknesses).
2. **A unified implementation.** Mixup, Global kNN, Global Distance and RegMix are all written as kNN mixing policies and run in one JAX code base, together with Manifold Mixup.
3. **A controlled evaluation.** Besides the standard comparison, we add controls the original study does not report: a random policy, a shuffled learned policy, sensitivity to the Beta parameter, variation across controller seeds, and paired significance tests.
4. **A calibrated conclusion.** RegMix has the lowest mean test error, but its margin over no augmentation is small, and several baseline gaps are within seed noise. The learned per-example policy is also not reproducible: controllers trained from different seeds disagree on individual assignments, though they reach similar error.

2 Related Work

Mixup and its variants. Mixup builds new training examples as convex combinations of two inputs and their labels, with the weight drawn from a Beta distribution (Zhang et al., 2018). Manifold Mixup applies the same interpolation to hidden representations (Verma et al., 2019). AdaMixup learns the mixing weight to avoid "manifold intrusion", where a mixed example falls in the region of a different label (Guo et al., 2019). CutMix and Puzzle Mix replace pixel interpolation with image-specific recombination (Yun et al., 2019; Kim et al., 2020). Theoretical work links Mixup to a regularizing effect on the model (Carratino et al., 2020; Zhang et al., 2021), and it can be read as a form of vicinal risk minimization (Chapelle et al., 2000).

Mixing for regression. Most of this literature targets classification. C-Mixup biases the choice of mixing partner toward examples with similar labels, which suits regression because label distance is meaningful (Yao et al., 2022). RegMix instead

restricts partners by input distance and learns, per example, how many nearest neighbours may be used (Hwang & Whang, 2021). We study the RegMix formulation and do not compare against C-Mixup, which is a limitation we return to in Section 8.

Learned augmentation policies. AutoAugment trains a recurrent controller with reinforcement learning to choose augmentation operations, rewarded by validation accuracy (Cubuk et al., 2019). It builds on neural architecture search with reinforcement learning (Zoph & Le, 2017) and uses proximal policy optimization (Schulman et al., 2017). RegMix borrows this loop but searches a different space: not which operation to apply, but how many neighbours each training example may mix with.

Machine learning for thin-film metrology. Reflectance spectra are a standard way to infer film thickness without destroying the sample, and learned models can map spectra to thicknesses. Recent work trains such models on theoretical spectra and studies augmentation for them (Lee & Jin, 2024). The competition dataset used by RegMix poses the multi-layer version of this task (DACON, 2020).

3 Method

We describe mixing as a per-example policy, then the controller that learns it, then two implementation choices that differ from the original paper.

3.1 kNN mixing policies

Let $D = \{(x_i, y_i)\}_{i=1}^{N}$ be the training set, with features standardised and Euclidean distance between inputs. A policy assigns each example i a number k_i from the option set $K = \{0\} \cup \{2^j : j = 0, \dots, 9\}$. If $k_i = 0$, example i is never used as the anchor of a mixed example. Otherwise a partner j is drawn uniformly from the k_i nearest neighbours of i, and the mixed example is

$$\begin{aligned} \tilde{x} &= \lambda x_i + (1-\lambda) x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda) y_j, \\ j &\sim \mathrm{Unif}\big(\mathrm{NN}_{k_i}(i)\big), \qquad \lambda \sim \mathrm{Beta}(\alpha, \alpha). \end{aligned} \tag{1}$$

With $\alpha = 100$, as in the original paper, λ stays close to 0.5 (its standard deviation is $1/\sqrt{4(2\alpha+1)} \approx 0.035$). Every baseline except Manifold Mixup is a special case of this formulation, which lets us run them through identical code (Table 1).

Manifold Mixup mixes hidden representations at a random layer instead and is trained separately.

3.2 Learning the policy

The controller is an MLP with four hidden layers of 100 units. Its input is a constant vector, and its output is one softmax over K for each of the N training examples. Each iteration follows the loop in the original paper's Algorithm 1:

1. Sample $T = 20$ policies from the controller.
2. For each policy, train a fresh regression model on D plus the mixed examples it produces.
3. Take the best validation loss L_{val} of each model and compute the reward below.
4. Update the controller with proximal policy optimisation (PPO).

The reward of the t-th sampled policy, with a moving-average baseline δ, is

$$R_t = \frac{1}{L_{\mathrm{val}}^{(t)}} - \delta, \qquad \delta \leftarrow 0.95\,\delta + 0.05\,\frac{1}{L_{\mathrm{val}}^{(t)}}. \tag{2}$$

The moving average δ is the reward baseline. The controller is updated by minimising the PPO objective

$$\begin{aligned} \mathscr{L}(\theta) = &-\frac{1}{TN}\sum_{t=1}^{T}\sum_{i=1}^{N} \min\Big(\rho_{t,i}\hat{A}_t,\ \mathrm{clip}(\rho_{t,i}, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big) \\ &- \beta\,\mathscr{H}(\pi_\theta), \\ \rho_{t,i} = &\frac{\pi_\theta(a_{t,i} \mid i)}{\pi_{\mathrm{old}}(a_{t,i} \mid i)}, \end{aligned} \tag{3}$$

where $a_{t,i} \in K$ is the option sampled for example i in policy t, $\hat{A}_t = (R_t - \bar{R})/\mathrm{std}(R)$ is the reward standardised within the iteration, $\varepsilon = 0.2$, $\beta = 0.01$, and $\mathscr{H}$ is the average entropy of the per-example softmax outputs. We use the settings stated in the original paper where it gives them: learning rate 0.002, entropy weight 0.01, and the Adam optimiser. The PPO clip range (0.2), the number of PPO passes per batch (4), and standardising the advantages within each batch are our own choices.

3.3 Two implementation differences

On-the-fly mixing. The original method builds the whole mixed set once per policy. We instead draw one fresh partner and one fresh λ for every anchor in every epoch. The expected distribution of mixed examples is the same, but the training cost is much lower and the model never sees the same mixed example twice.

Proxy model for the controller. Training 20 full-size models per iteration is too slow on our hardware (2 CPU cores). The controller and the baseline hyper-parameters are therefore tuned against a smaller MLP ($[226, 256, 128, 4]$, 40 epochs). Final results use the original paper's architecture. Section 8 discusses what this costs.

4 A Physics-Simulated Thin-Film Dataset

The dataset used by RegMix comes from a public competition on predicting the thicknesses of a multi-layer semiconductor film from its reflectance spectrum (DACON, 2020). Access requires registering for the competition with a mobile phone number, and the verification step rejects non-Korean numbers, so we could not obtain it. We built a simulator that reproduces

Method	Rule for k_i	What is tuned
Vanilla	$k_i = 0$ for all i	nothing
Mixup	$k_i = N-1$ for all i (any pair)	Beta parameter α
Global kNN	the same k for all i	k
Global Distance	number of neighbours within distance d of i	d
RegMix	learned separately for each i	controller (Section 3.2)

Table 1: Every mixing baseline except Manifold Mixup is a special case of the kNN-policy formulation in Eq. (1).

its shape and physics instead. It is a stand-in, not a copy: the optical constants and wavelength grid below are our own choices.

Device and physics. The stack is air / Si_3N_4 / SiO_2 / Si_3N_4 / SiO_2 / silicon substrate (Figure 1). This is the layer order listed on the competition's data page (DACON, 2020); the RegMix paper itself does not state it. Light arrives at normal incidence. For layer j with complex refractive index N_j and thickness d_j, the phase thickness and characteristic matrix are

$$\delta_j = \frac{2\pi N_j d_j}{\lambda}, \qquad M_j = \begin{pmatrix} \cos\delta_j & \frac{i\sin\delta_j}{N_j} \\ iN_j\sin\delta_j & \cos\delta_j \end{pmatrix}. \quad (4)$$

Multiplying the four matrices and applying the substrate index N_s gives the amplitude reflection coefficient r and the reflectance R (with air index $N_0 = 1$):

$$\begin{pmatrix} B \\ C \end{pmatrix} = M_1 M_2 M_3 M_4 \begin{pmatrix} 1 \\ N_s \end{pmatrix}, \qquad r = \frac{N_0 B - C}{N_0 B + C}, \qquad R(\lambda) = |r|^2. \quad (5)$$

Reflectance is evaluated at 226 wavelengths evenly spaced between 400 and 800 nm.

Optical constants. SiO_2 follows the Malitson Sellmeier fit for fused silica,

$$n^2_{SiO_2}(\lambda) = 1 + \sum_{m=1}^{3} \frac{B_m \lambda^2}{\lambda^2 - C_m^2}, \quad (6)$$

with λ in micrometres, $B = (0.6961663, 0.4079426, 0.8974794)$ and $C = (0.0684043, 0.1162414, 9.896161)$. Si_3N_4 uses a lossless Cauchy model,

$$n_{Si_3N_4}(\lambda) = 1.90 + \frac{1.5\times 10^4}{\lambda^2} \qquad (\lambda \text{ in nm}), \quad (7)$$

which gives about 1.95 at 550 nm. Silicon uses a small table of approximate n and k values at 50 nm steps, linearly interpolated. The Si_3N_4 and silicon values are textbook-style approximations that we did not fit to a measured reference.

Sampling and splits. Each of the four thicknesses is drawn independently and uniformly from 10 to 100 nm, the range

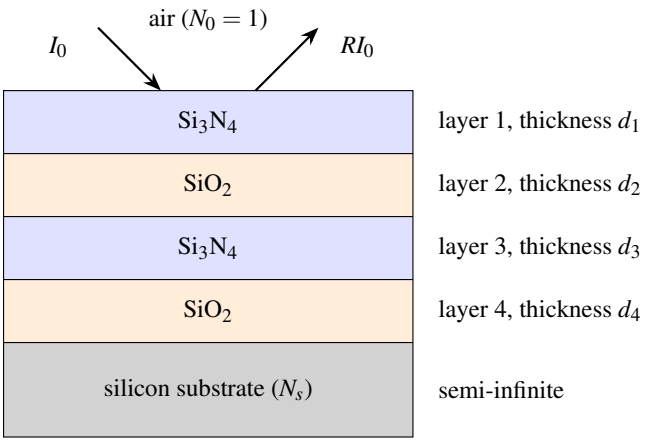


Figure 1: The simulated device: four thin films of thickness $d_1, \ldots, d_4 \in [10, 100]$ nm on a silicon substrate, probed by light at normal incidence (drawn at an angle for clarity). The reflectance $R(\lambda)$ at 226 wavelengths forms the feature vector, and the four thicknesses are the labels.

the RegMix paper uses for its Synthetic subset. We simulate a pool of 20,000 stacks with seed 42 and draw disjoint splits from it: 600 for training, 400 for validation and 1,000 for testing. Every example has 226 reflectance features and 4 thickness labels, matching the original paper's Table 1. We add no measurement noise. Figure 2 shows example spectra.

Checks. Five automated tests pass:

- Reflectance stays within $[0, 1]$ and has no missing values.
- With all thicknesses set to zero, the result equals the Fresnel reflectance of air on silicon.
- A single SiO_2 layer on silicon matches the closed-form thin-film formula to 10^{-10}.
- Spectra change measurably when any thickness changes.
- The saved splits have the right sizes and no overlap.

As a learnability check, a small scikit-learn MLP trained on the 600 training examples reaches a validation RMSE of 14.6 nm. Always predicting the training mean gives 25.8 nm, so the spectra carry usable signal about thickness.

What this dataset cannot tell us. The competition's wavelengths are anonymised and its simulator settings are unknown,

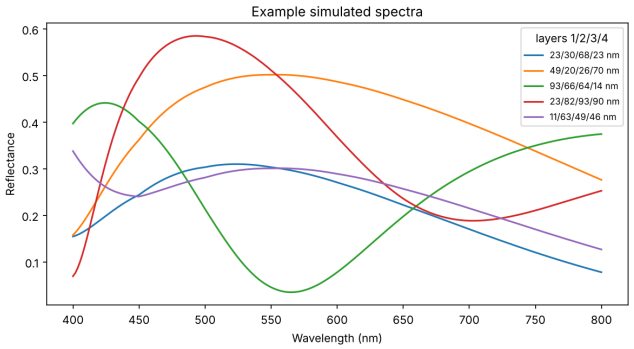


Figure 2: Five simulated reflectance spectra. The legend gives the thicknesses $d_1/d_2/d_3/d_4$ in nm. Different stacks produce clearly different interference patterns, which is what makes thickness recoverable from the spectrum.

so absolute error values here are not comparable with the original paper. Because the data is noise-free and generated from smooth physics, it may also be easier or harder to augment than real measurements. Only the relative comparison between methods transfers.

5 Experimental Setup

Regression model. All final results use one architecture, an MLP with hidden layers of 2048, 1024, 512 and 256 units, each followed by LayerNorm and ReLU, and a linear output of 4 thicknesses. Inputs and labels are standardised with training-set statistics. Training uses Adam (learning rate 0.001), batch size 64 and 100 epochs, and we keep the checkpoint with the lowest validation loss.

Methods compared. Vanilla (no augmentation), Mixup, Manifold Mixup, Global kNN, Global Distance and RegMix. We omit AdaMixup: it relies on a discriminator designed for class labels, and the original paper does not describe a regression version, so any implementation would be our invention.

Hyper-parameter selection. All tuning uses validation loss only, measured on the small proxy model ($[226, 256, 128, 4]$, 40 epochs), averaging 20 models per candidate with identical random keys across candidates (Table 2).

The original Mixup default ($\alpha = 1$) is also evaluated. The selected $\alpha = 0.02$ for both Mixup and Manifold Mixup is the smallest value in the grid, so the true optimum may lie below it. We did not extend the grid, and we list this as a limitation in Section 8. We used a grid in place of the Bayesian optimisation the original describes, since each search is one-dimensional.

Evaluation protocol. Each method is trained with 5 seeds, using the same seeds across methods, and scored on the 1,000 held-out test examples. The test set is never used for any choice. Error is RMSE in nanometres, the square root of the mean squared error over all examples and all four layers:

$$\mathrm{RMSE} = \sqrt{\frac{1}{4N_{\mathrm{te}}} \sum_{i=1}^{N_{\mathrm{te}}} \sum_{l=1}^{4} \left(\hat{y}_{il} - y_{il}\right)^2}\,. \tag{8}$$

We report mean and standard deviation across seeds, and test each method against RegMix with a paired t-test over seeds. Section 7 adds five more seeds, controls and sensitivity analyses.

Compute. Everything runs on 2 CPU cores with JAX, with no GPU. Training the 20 proxy models of one controller iteration takes about 7 seconds, and one full-size model takes about 40 seconds.

6 Main Results

RegMix has the lowest mean test RMSE, 13.77 nm, but its lead over no augmentation is only 0.14 nm (about 1%). The original paper reports 15.68 for Vanilla and 13.49 for RegMix on its Synthetic data, a 14% gap; absolute values differ because our data is different.

Three observations follow from Table 3.

1. **The ranking is partly noise.** The seed-to-seed standard deviation (0.12 to 0.24 nm) is as large as most gaps. Only the comparison with tuned Mixup reaches $p < 0.05$, and with six comparisons it does not survive a Bonferroni correction (threshold 0.008).

2. **Our pattern differs from the original on Synthetic.** In our data neither Mixup nor Manifold Mixup beats Vanilla, and the three neighbourhood-limited methods beat it only by small margins. The original paper reports something different (Table 4): there Mixup (13.84) improves on Vanilla (15.68) by almost 2 nm, and Global Distance (14.09) is worse than Mixup. We see the opposite on both points: Mixup is no better than Vanilla, and Global Distance (13.82) is better than Mixup (13.99 or 14.14). What does agree is that RegMix has the lowest mean error, and that Global kNN is ahead of Manifold Mixup.

3. **RegMix's validation advantage overstates its test advantage.** RegMix leads Vanilla by 0.38 nm on validation but by 0.14 nm on test. This is expected, because the controller was optimised against the validation set, and it is why we report test error as the headline number.

The gap comes from Vanilla, not from RegMix. Our RegMix (13.77) is close to the original's (13.49). Our Vanilla (13.91) is far better than theirs (15.68), and that difference accounts for almost all of the gap between the two papers' results. Apart from the original Vanilla, every method in both papers lands between 13.5 and 14.1 nm. This looks like a performance floor that augmentation does not get below on this task. Our data do not tell us why. The floor cannot be measurement noise, since our spectra are noise-free, so a limit

Method	Searched	Selected
Mixup	$\alpha \in \{0.02, 0.05, 0.1, 0.2, 0.5, 1, 2\}$	0.02
Manifold Mixup	same grid	0.02
Global kNN	k, 18 values from 1 to 512	8
Global Distance	d, 14 values from 0.05 to 1.0 of the largest pairwise distance	0.1
RegMix	controller trained for 300 iterations	argmax policy

Table 2: Hyper-parameter search and the selected values.

Method	Test RMSE (nm)	Val RMSE (nm)	RegMix better by (nm)	Paired t-test p
RegMix	$\mathbf{13.772 \pm 0.124}$	13.508	–	–
Global Distance ($d = 0.1$)	13.824 ± 0.141	13.622	0.052	0.616
Global kNN ($k = 8$)	13.876 ± 0.244	13.692	0.103	0.464
Vanilla	13.908 ± 0.157	13.887	0.136	0.108
Mixup ($\alpha = 1$, original default)	13.986 ± 0.165	13.860	0.214	0.147
Manifold Mixup ($\alpha = 0.02$)	14.004 ± 0.193	13.937	0.231	0.157
Mixup ($\alpha = 0.02$, tuned)	14.136 ± 0.153	13.908	0.363	0.013

Table 3: Test RMSE (mean $\pm$ std) over five seeds, 1,000 test examples. Rows are sorted by test RMSE. Standard deviations here and in every other table use $n - 1$.

set by the 600-example training set and the thickness range is more likely. A weaker or less well-regularised Vanilla baseline in the original setup would also fit. We cannot test either explanation without the original data.

Baseline tuning on the proxy model also did not transfer cleanly. Mixup with $\alpha = 0.02$ had the best proxy validation loss of its grid but the worst full-model test error. Section 7 tests how much of RegMix's remaining edge comes from its learned structure, as opposed to simply mixing a moderate amount.

7 Additional Analyses

The original study reports one comparison table. We add four analyses to see which parts of the RegMix gain are reliable.

7.1 Ten seeds and multiple-comparison correction

We trained every method on five further seeds (5 to 9) and pooled them with the main run, giving ten paired seeds per method. Table 5 reports the difference to RegMix with a bootstrap confidence interval, the paired t-test p-value, and a Holm-adjusted p-value that accounts for the $m = 6$ comparisons. With the raw p-values sorted ascending, $p_{(1)} \le \cdots \le p_{(m)}$, the Holm-adjusted value is

$$\tilde{p}_{(i)} = \min\Big(1,\ \max_{j \le i} (m - j + 1)\, p_{(j)}\Big). \tag{9}$$

The extra seeds leave the ranking nearly unchanged (RegMix 13.77 on the first five seeds, 13.74 on the last five), which suggests the five-seed result was not a fluke. What the larger sample supports is narrower than the headline ranking:

- **RegMix reliably beats unrestricted Mixup.** Both Mixup settings are significantly worse after correction (Holm p of 0.027 and 0.005).
- **RegMix over no augmentation is suggestive, not established.** The interval for the 0.14 nm gain excludes zero (0.04 to 0.25 nm) and RegMix wins on 8 of 10 seeds, but the corrected p-value is 0.155.
- **RegMix is indistinguishable from the best simple baseline.** Global Distance with a tuned threshold is within 0.03 nm, and RegMix wins on only 5 of 10 seeds.

7.2 Does the learned assignment matter?

A fair worry is that RegMix helps only because it mixes a moderate amount, which any policy with a similar spread of neighbourhood sizes would also do. We test this with three controls, each trained on the same five seeds as the main run:

- **Random policy.** Every example gets a neighbourhood size drawn uniformly from the 11 options, with no learning.
- **Shuffled RegMix policy.** The learned neighbourhood sizes are randomly reassigned to different examples, which keeps their overall distribution and destroys the per-example match.
- **Sampled RegMix policy.** Instead of the most likely option, each example's size is sampled from the controller's output.

The random policy is no better than no augmentation (13.94 against 13.91 nm), and it is significantly worse than the learned

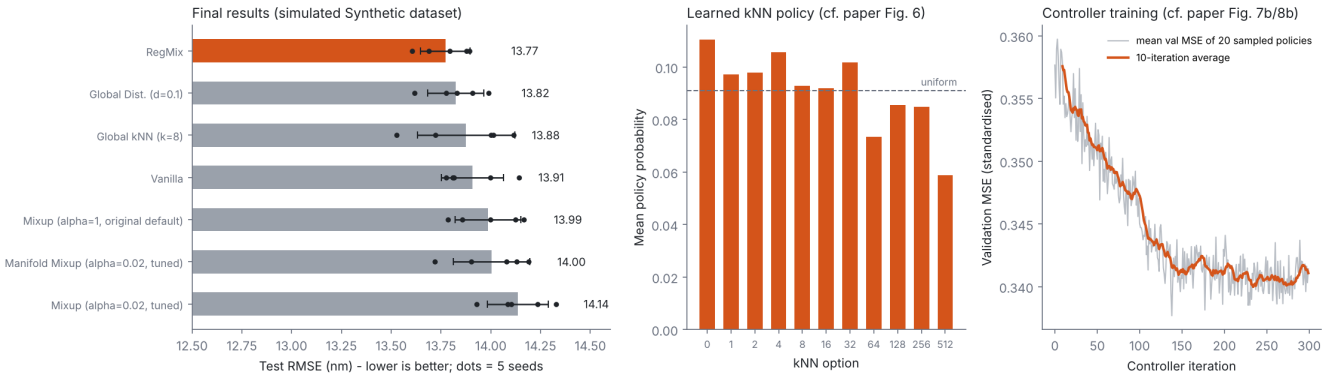


Figure 3: Left: test RMSE per method for the main five seeds (dots are individual seeds, bars show the mean with one standard deviation, $n-1$; the axis starts at 12.5 nm, so bar lengths are not proportional to error). Middle: the average probability the trained controller assigns to each neighbourhood size, against a uniform policy. Right: validation loss of the 20 sampled policies over 300 controller iterations.

Method	Original paper, Synthetic (RMSE)	Ours, simulated (RMSE, nm)
Vanilla	15.684 ± 0.279	13.908 ± 0.157
Mixup	13.836 ± 0.091	13.986 ± 0.165 ($\alpha = 1$); 14.136 ± 0.153 (tuned)
Manifold Mixup	14.046 ± 0.107	14.004 ± 0.193
Global kNN	13.759 ± 0.049	13.876 ± 0.244
Global Distance	14.086 ± 0.088	13.824 ± 0.141
RegMix	13.494 ± 0.091	13.772 ± 0.124

Table 4: Original paper's Table 3 (Synthetic rows, as printed; AdaMixup, 13.760, is not implemented here) against our five-seed results. The datasets differ, so absolute values are not comparable, but the pattern across methods can be.

policy ($p = 0.013$). The shuffled policy falls between the two and is not statistically separable from either, so these five seeds cannot say how much of RegMix's gain comes from the distribution of sizes and how much from their placement. What the controls do show is that mixing a moderate amount at random is not enough, and that the benefit survives sampling from the controller, so it does not hinge on taking the single most likely option.

These tests have limited power, and the p-values above are not corrected for the several comparisons in this section.

7.3 Sensitivity to the Beta parameter

The mixing weight is drawn from $\mathrm{Beta}(\alpha, \alpha)$, whose standard deviation is $1/\sqrt{4(2\alpha+1)}$. A small α gives weights near 0 or 1 (little mixing; $\alpha = 1$ is uniform, with standard deviation 0.29), and a large α pins them near 0.5, an average of the pair (0.035 at $\alpha = 100$ and 0.011 at $\alpha = 1000$). All main results use $\alpha = 100$. We keep the learned RegMix policy fixed and retrain the final models with other values, on the same five seeds.

Every setting beats Vanilla on average, and the difference between settings (about 0.2 nm from best to worst) is the same size as the seed noise. The largest α does best and the smallest does worst, though the middle values are not ordered ($\alpha = 10$ is slightly ahead of $\alpha = 100$). Validation and test error rank the largest and smallest settings the same way, so the pattern is not an artefact of looking at the test set. The best setting, $\alpha = 1000$, is 0.28 nm (2%) better than Vanilla with an uncorrected p of 0.004, but we did not choose α in advance, so we treat this as exploratory. Because $\alpha = 1000$ stands out only after looking at the results, its advantage is likely inflated by selection (the winner's curse), and it should not be read as an estimate of what the method can reach. At most it hints that averaging neighbouring pairs may suit these smooth spectra. Confirming that would need a pre-specified α, fresh seeds, and a controller retrained at each value.

7.4 Is the learned policy reproducible?

The main RegMix result comes from one controller run. We trained two more controllers from different random initialisations (seeds 1 and 2, each for 200 iterations to save time) and evaluated each one's final policy on the same five regression seeds.

All three controllers beat Vanilla on average, but by different amounts (0.08 to 0.26 nm), and only seed 2 reaches $p < 0.05$ uncorrected. Their overall behaviour is similar: each one mixes 89% of the examples with a mean neighbourhood size between 63 and 75.

Method	Test RMSE (nm)	RegMix better by (95% CI)	p	Holm p	RegMix lower in
RegMix	$\mathbf{13.758 \pm 0.123}$	–	–	–	–
Global Distance ($d = 0.1$)	13.786 ± 0.139	0.028 [−0.085, 0.138]	0.646	0.646	5 of 10
Global kNN ($k = 8$)	13.872 ± 0.178	0.114 [−0.022, 0.255]	0.162	0.323	7 of 10
Vanilla	13.897 ± 0.128	0.139 [0.036, 0.247]	0.039	0.155	8 of 10
Manifold Mixup ($\alpha = 0.02$)	13.936 ± 0.177	0.178 [0.020, 0.323]	0.059	0.178	8 of 10
Mixup ($\alpha = 1$)	14.007 ± 0.190	0.249 [0.121, 0.373]	0.005	0.027	8 of 10
Mixup ($\alpha = 0.02$, tuned)	14.184 ± 0.220	0.425 [0.271, 0.593]	0.001	0.005	10 of 10

Table 5: Ten paired seeds per method. Standard deviations use $n - 1$. Differences are paired by seed.

Policy	Mean k	Share mixed	Test RMSE (nm)	p vs Vanilla	p vs RegMix
Vanilla	0	0%	13.908 ± 0.157	–	0.108
Random policy	93.0	91%	13.937 ± 0.081	0.688	0.013
Shuffled RegMix policy	71.1	89%	13.873 ± 0.156	0.620	0.292
RegMix (most likely option)	71.1	89%	$\mathbf{13.772 \pm 0.124}$	0.108	–
RegMix (sampled from controller)	73.6	89%	13.758 ± 0.111	0.008	0.838

Table 6: Controls over five seeds; standard deviations use $n - 1$. The mean k for the sampled policy is the expected value under the controller, and each seed's actual draw differs slightly. The p-value of 0.108 for RegMix versus Vanilla is the main-run value from Table 3.

The individual assignments are another matter. We compared the three policies example by example (Table 9):

The three controllers agree on which example gets which neighbourhood size only 0.8 to 1.4 percentage points more often than chance. With 600 examples, the standard error of an agreement rate near 9% is about 1.2 points, so this is within about one standard error of chance, and the sizes are essentially uncorrelated. This is not simply uncertainty. Seeds 0 and 2 are each confident (their chosen option has a mean probability of 0.69 and 0.71; seed 1 is less so, at 0.35), yet those two agree on only 10.7% of examples, against 9.3% by chance. The per-example assignment therefore looks arbitrary: different controllers settle on different, equally good policies. What they share is the overall distribution of sizes. Together with the shuffled-policy control in Section 7.2, this indicates that on this data RegMix's benefit comes from how much mixing it does and with what range of neighbourhoods, not from finding which particular examples should mix widely. We cannot rule out per-example structure that a larger training set or noisier data would reveal.

8 Discussion and Limitations

What the evidence supports. On ten paired seeds RegMix beats both Mixup settings after multiple-comparison correction, and the neighbourhood-limited methods all sit slightly ahead of no augmentation, though not significantly. This is consistent with the idea behind RegMix, that mixing nearby examples is safer than mixing any pair. It does not reproduce the original's Synthetic pattern: there Mixup improves strongly on Vanilla and Global Distance is worse than Mixup, while here Mixup does not help and Global Distance is better.

What it does not support. The size of the original gain (14% over Vanilla) was not reproduced, because our Vanilla is much stronger than theirs (Section 6); we see about 1%, and its corrected significance against Vanilla is weak. RegMix is also not distinguishable from a Global Distance baseline with one tuned threshold. The case for a per-example learned policy is the weakest part: three controllers trained from different seeds agree on individual assignments no more than chance, a random assignment with a similar spread of sizes (the shuffled control) is not clearly worse, and the shuffled and learned policies cannot be separated with five seeds. The more modest reading of our data is that RegMix finds a good overall distribution of neighbourhood sizes.

Why our gain may be smaller. Section 6 shows that the gap comes from a stronger Vanilla baseline and not from a weaker RegMix, and that nearly all methods in both papers sit near a common floor. Beyond that we cannot say why. Possible reasons include that our simulated spectra are smooth and noise-free, so Vanilla training is already well regularised; that our regression model uses LayerNorm and a best-validation checkpoint, which may already capture some of what augmentation provides; and that we trained the controller on a proxy model. Only access to the original data could separate these.

Selection and leakage. Validation data was used for baseline tuning, for the controller's reward and for choosing checkpoints, so validation error is optimistic for RegMix (13.51 against 13.76 on test). The test set was not used for any choice in Sections 5 and 6. The α sensitivity in Section 7.3 and the controls in Section 7.2 are exploratory, and their p-values are uncorrected.

Proxy transfer. Hyper-parameters for the baselines and the controller were found on a small model and applied to a larger one. Tuned Mixup ($\alpha = 0.02$) won on the proxy and lost on

α	Val RMSE (nm)	Test RMSE (nm)	Better than Vanilla by (nm)	p vs Vanilla	p vs $\alpha = 100$
1	13.570	13.818 ± 0.102	0.091	0.226	0.140
10	13.468	13.706 ± 0.108	0.203	0.027	0.492
100 (main setting)	13.508	13.772 ± 0.124	0.136	0.108	–
1000	13.422	13.628 ± 0.072	0.280	0.004	0.081

Table 7: Five seeds, standard deviations use $n-1$. Vanilla is 13.908 nm on test and 13.887 nm on validation. The policy was learned at $\alpha = 100$ and not retrained for the other values.

Controller	Iter.	Final entropy (nats)	Mean k	Share mixed	Val RMSE (nm)	Test RMSE (nm)	p vs Vanilla
Seed 0 (main)	300	1.03	71.1	89%	13.508	13.772 ± 0.124	0.108
Seed 1	200	1.92	75.1	89%	13.603	13.829 ± 0.091	0.117
Seed 2	200	0.96	63.2	89%	13.458	13.650 ± 0.141	0.042
Vanilla	–	–	0	0%	13.887	13.908 ± 0.157	–

Table 8: Standard deviations use $n-1$. Mean k and share mixed describe the policy actually used (most likely option per example). The three controllers average 13.75 nm on test, 0.16 nm below Vanilla.

Controller pair	Same option	Chance level	Spearman ρ of k
Seed 0 and seed 1	10.2%	9.3%	0.013
Seed 0 and seed 2	10.7%	9.3%	0.044
Seed 1 and seed 2	10.2%	9.4%	0.073

Table 9: Chance agreement is $\sum_a p_a q_a$, where p_a and q_a are the shares of examples each controller assigns to option a. It is computed from each pair's actual option shares, and is slightly above the $1/11 \approx 9.1\%$ of two uniform policies.

the full model, so the baselines may be under-tuned. The same proxy mismatch applies to RegMix, so the comparison is fair in procedure, but not necessarily at each method's best.

Scope. One simulated dataset of 600 training examples, one architecture, controllers of 200 to 300 iterations on two CPU cores. We did not compare against AdaMixup or C-Mixup, and the simulated data has no measurement noise. The tuned Mixup and Manifold Mixup values ($\alpha = 0.02$) lie at the edge of the searched grid, so those two baselines may be under-tuned. Conclusions about real spectra, or about larger or noisier datasets, would need new experiments.

9 Conclusion

On a physics-simulated thin-film regression task, RegMix reaches 13.76 nm test RMSE over ten paired seeds, against 13.90 nm with no augmentation and 14.01 to 14.18 nm with unrestricted Mixup. The improvement over Mixup is statistically solid, the improvement over no augmentation is about 1% and does not survive multiple-comparison correction, and a single tuned neighbourhood threshold performs about as well. The original paper's Synthetic pattern, in which Mixup clearly beats Vanilla, did not appear here, because our Vanilla baseline is far stronger than the original's (13.9 against 15.7 nm) while RegMix lands close to the original value.

The controls complicate the story. A random policy gives no benefit, but three controllers trained from different seeds produce uncorrelated per-example assignments with similar error, so what RegMix learns here is an overall distribution of mixing ranges and not a per-example rule. We release the simulator, the JAX implementation and every experiment script, so the comparison can be rerun on the original data, on noisier spectra or with retrained controllers at other Beta parameters.

References

Carratino, L., Cissé, M., Jenatton, R., & Vert, J.-P. (2022). On mixup regularization. *Journal of Machine Learning Research*, 23 (paper 20-1385). arXiv:2006.06049.

Chapelle, O., Weston, J., Bottou, L., & Vapnik, V. (2000). Vicinal risk minimization. *Advances in Neural Information Processing Systems 13*, 416–422.

Cubuk, E. D., Zoph, B., Mane, D., Vasudevan, V., & Le, Q. V. (2019). AutoAugment: Learning augmentation strategies from data. *CVPR*, 113–123.

DACON Co., Ltd. (2020). Challenge on semiconductor thin film thickness analysis (data page). `dacon.io/competitions/official/235554/data`.

Guo, H., Mao, Y., & Zhang, R. (2019). MixUp as locally linear out-of-manifold regularization. *AAAI*, 3714–3722.

Hwang, S.-H., & Whang, S. E. (2021). RegMix: Data mixing augmentation for regression. arXiv:2106.03374 (v4 used here).

Kim, J.-H., Choo, W., & Song, H. O. (2020). Puzzle Mix: Exploiting saliency and local statistics for optimal mixup. *ICML*, 5275–5285.

Lee and Jin (2024). Thin film thickness analysis based on a deep learning algorithm using data augmentation. *Proceedings of SPIE*, 12955, 129552G. doi:10.1117/12.3010091.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

Verma, V., Lamb, A., Beckham, C., Najafi, A., Mitliagkas, I., Courville, A., Lopez-Paz, D., & Bengio, Y. (2019). Manifold mixup: Better representations by interpolating hidden states. *ICML*.

Yao, H., Wang, Y., Zhang, L., Zou, J., & Finn, C. (2022). C-Mixup: Improving generalization in regression. *NeurIPS*. arXiv:2210.05775.

Yun, S., Han, D., Chun, S., Oh, S. J., Yoo, Y., & Choe, J. (2019). CutMix: Regularization strategy to train strong classifiers with localizable features. *ICCV*, 6022–6031.

Zhang, H., Cissé, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond empirical risk minimization. *ICLR*.

Zhang, L., Deng, Z., Kawaguchi, K., Ghorbani, A., & Zou, J. (2021). How does mixup help with robustness and generalization? *ICLR*.

Zoph, B., & Le, Q. V. (2017). Neural architecture search with reinforcement learning. *ICLR*.

A Hyper-parameters and Reproducibility

Component	Setting
Final regression model	MLP $226 \to 2048 \to 1024 \to 512 \to 256 \to 4$, LayerNorm + ReLU, Adam, learning rate 0.001, batch 64, 100 epochs, best-validation checkpoint
Proxy model (tuning, controller)	MLP $226 \to 256 \to 128 \to 4$, LayerNorm + ReLU, Adam, learning rate 0.001, batch 64, 40 epochs
Mixing	Per epoch, one partner and one λ per anchor with $k_i > 0$, so an epoch holds N + (number of mixed anchors) examples; $\alpha = 100$ for kNN-based methods
kNN options K	0, 1, 2, 4, 8, 16, 32, 64, 128, 256, 512
Controller network	4 hidden layers of 100 units (ReLU), constant input of ones, output of $N \times 11$ logits, output layer initialised with small weights so the starting policy is near uniform
Controller training	PPO, Adam learning rate 0.002, clip 0.2, 4 passes per iteration, entropy weight 0.01, $T = 20$ policies per iteration, 300 iterations, advantages standardised per iteration
Reward	$1/(\text{best validation MSE})$ minus an exponential moving average of past rewards (decay 0.95)
Final RegMix policy	Most likely option for each training example
Data	20,000-stack pool (seed 42); 600 train, 400 validation, 1,000 test; thicknesses uniform in 10 to 100 nm; 226 wavelengths from 400 to 800 nm; no noise
Seeds	Regression models use seeds $1000 + s$; proxy tuning uses a fixed set of 20 keys; main controller seed 0

Table 10: Hyper-parameters.

Item	Status
Code	Single JAX code base: simulator (`thinfilm.py`), data generation, policies and trainers (`regmix.py`), tuning, controller, final evaluation, extra experiments, and report scripts
Hardware	2 CPU cores, 7 GB memory, no GPU
Run time	Tuning about 20 minutes; one 300-iteration controller about 35 minutes; one full-size model about 40 seconds
Validation use	Used for baseline tuning, controller reward and checkpoint choice; never the test set
Original data	Not used; see Section 4
Not reproduced	AdaMixup; Bayesian optimisation for baseline tuning; the original paper's other datasets
Verified	Simulator tests pass; the controller run is deterministic given its seed (the first 150 iterations of the 300-iteration run reproduce the earlier 150-iteration run)

Table 11: Reproducibility checklist.